# Ellis Playbook

**How to run Ellis today: which command answers which question, what to give it, how to read what comes back, and what to do next. Everything here works now; the agent stages still planned are named at the end.**

**As of** 8 Nov 2026 **For** K.D. and whoever operates Ellis **Prepared by** Quincy Nkwain Ninying, Forward Deployed Engineer

**Repository** `github.com/qninying/platform-reliability-agent`

| 40 | 16 | 3 | 4 | 0 |
|---|---|---|---|---|
| commands that answer a reliability question | sandbox rehearsals, one per rung built | kinds of input: a platform checkout, a dated export, a sandbox fixture | verdicts, each with its own next step | commands that can write to the platform |

BEFORE YOU RUN ANYTHING

## Set up once, then give each command the input it reads

```
git clone https://github.com/qninying/platform-reliability-agent
cd platform-reliability-agent
npm ci          # Node 23.6 or later; no runtime dependencies
npm test        # 893 tests; every one should pass before you trust an answer
```

### A platform checkout

`--source <folder>`

A read-only clone of the platform repository. Ellis reads its code and never changes it. Note the commit you cloned at: answers name it.

### A dated export

`--snapshot <file.json>`

Records a person with admin access pulls from the platform's admin endpoints, stamped with when and by whom. Ellis has no production access; an answer is about that export, at that time.

### A sandbox fixture

`--fixture`, `--sandbox`

For rehearsals only. Point `--sandbox` at an empty folder; nothing outside it is written. The rungs started by a person also need `--operator <name>`.

Each command prints what it needs at the top of its file in `src/cli/`, and refuses with exit 1 and the missing piece if you leave one out. Run any of them as `npm run <command> -- <arguments>`.

WHICH COMMAND ANSWERS WHICH QUESTION

## Start from the question, not the tool

Each command is one capability's check. Its document in `docs/capabilities/` says what it reads, what it cannot see, and its exit codes.

| WHEN THE QUESTION IS | RUN |
|---|---|
| **Is anything broken right now?** | `cron-health` `agent-health` `orchestration-health` `scheduler-liveness` `dead-letters` `stale-actions` |
| **What ran, when, and did it fail?** | `cron-runs` `incident-trace` `error-class-coverage` `dead-letter-triage` `replay-harness` |
| **Is the registry telling the truth?** | `registry-reconcile` `registry-ask` `timer-inventory` `default-on-timers` `schedule-governance` `overlap-guard` |
| **Who can do what, and is it safe?** | `security-posture` `access-watch` `behavior-watch` `security-rollup` `manager-access` `autonomy-authority` |
| **Can we recover, and is the data kept as promised?** | `backup-position` `schema-drift` `retention-sweep` `preview-reaping` `resilience-posture` |
| **Is this change ready, and what should be done about an agent?** | `deploy-readiness` `release-evidence` `promotion-evidence` `bug-baseline` `disable-recommendation` `ticket-resolution` `orchestration-repair` |
| **What does K.D. need to see or decide?** | `reliability-report` `reliability-thresholds` `evolution-proposals` `technology-direction` `prompt-performance` |

The briefing, `reliability-report`, composes several of these into one text a person can read with no login, the unmeasured section first.

READING AN ANSWER

## Two layers: the command's own answer, and one shared envelope

On the command line, each command prints its own answer in its own words (`healthy`, `alerting`, `traced` and so on): a statement, the records it read, and what it cannot say. Every check also has an adapter to one shared envelope in `src/output/`, the shape the broker will call. Its fields:

| FIELD | WHAT IT TELLS YOU |
|---|---|
| `verdict` | **ok**, **problem**, **partly_unknown** or **unknown**. Read it first; the next section says what to do with each. |
| `evidence` | The row ids the verdict rests on. A problem or an ok with none cannot leave the command. |
| `blind_spots` | What the command could not see. Read these before you trust an ok. |
| `severity, severity_rule` | Set only from K.D.'s table. Until it has rules, every severity is unassigned, and the rule field says why. |
| `platform_signal` | A severity or alert the platform itself raised, shown as evidence, never used as Ellis's severity. |
| `as_of` | When the input was captured and by whom, or the sandbox fixture's name, marked "not production". |

Exit codes: **0** means the command answered, and for most commands that there is nothing to act on; **1** is always bad input, with an error class (`SourceUnavailable` when a file could not be read, `ContractViolation` when its shape is wrong). Other codes are per command and listed in its document.

WHAT TO DO ON EACH VERDICT

## Four verdicts, four next steps

Whatever words a command uses, its answer maps to one of these four, and the next step is the same.

### ok

1. Check the blind spots are empty.
2. Record it with its `as_of`: it is true of those rows, at that time.
3. Do not stretch it to rows or days it did not read.

### partly_unknown

1. Read each blind spot.
2. Decide whether what was seen is enough for the question.
3. If not, get the missing export and run again.

### unknown

1. Never read it as clean.
2. Fix the input: re-export, or re-clone the checkout.
3. If the record does not exist on the platform, that absence is the finding: take it to K.D.

### problem

1. Take it to K.D. with the row ids and the command's statement.
2. K.D. judges severity until the table has rules.
3. Do not change the platform yourself: a fix ships through a reviewed deploy by a person.

RUNNING A SANDBOX REHEARSAL

## Prove a mechanism on a copy, never on the platform

The 16 rehearsals end in `-sandbox`. Each runs a mechanism beside a control that must fail, and says whether every check held.

```
npm run restore-sandbox -- --fixture examples/backup/restore.fixture.json \
  --sandbox "$(mktemp -d)" --operator "<your name>"
```

### Exit 0: verified

Every check held, including the control. The statement ends "Sandbox only; nothing ran on the platform."

### Exit 3: not verified

The statement names the checks that failed. Read them: a rehearsal that fails is telling you something about the fixture or the mechanism.

### What it does not prove

Each answer lists its limits. A verified rehearsal is a proposal for K.D. and A.M., not a change to the platform.

WHO TO TAKE WHAT TO

## Ellis reports; people decide

| TAKE THIS | TO |
|---|---|
| A problem verdict, a severity question, a threshold, a decision on a timer or a scoring model, or a rung from R0 to R2 | K.D. |
| A governance question, a rung from R3 up, the gate and broker, or a change to the platform's plan | A.M. |
| A fix to the platform's code, data or timers | a person, through a reviewed deploy |

**Stop and report, never work around:**

- a command that tries to write outside its sandbox folder, or to the platform
- a test that fails after `npm ci`: do not trust any answer until it passes
- a check pointed at a production database: none is built to accept one, and production access is default-denied

WHEN THE AGENT STAGES LAND

## What changes, and what stays

### Changes

Cases start on their own: a job out of retries, a scheduled sweep, or K.D.'s question. The gate decides whether one may run, and the broker calls these same commands as the agent's tools.

### Stays

The envelope, the four verdicts and what to do on each; verdicts and severity from code and K.D.'s table, never from the model; nothing written to the platform.

### Not yet built

The entry points, gate, agent core, broker, report and audit log. Who builds them is open with A.M. This playbook gains a section when they land.

Sources: `package.json` (the 56 commands); the usage line at the top of each file in `src/cli/`; the output contract in `src/output/toolOutput.ts` and decision record 006; the capability documents in `docs/capabilities/`. Companion documents: Ellis Workflow, Ellis Architecture, Ellis Executive Brief.